

2026-07-01-auth-pivot-ssh-key

Auth pivot — SSH-key challenge-response login (supersedes PAM login)

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Supersedes:** the “live PAM login” part of 2026-07-01-real-account-code-server-design.md §3.1/§4. Everything else in that spec (host-native code-server AS milosvasic, editor projects-jail, Caddy forward-auth gate, fail-closed, SSH keys/.bashrc/binaries native, TLS modes) STANDS UNCHANGED.

Why the pivot (captured facts, §11.4.6 — not guessing)

This host is ALT Linux with the **tcb** shadow scheme: - /etc/tcb = drwx--x--- root shadow; per-user /etc/tcb/<user>/shadow is NOT readable by the user. - The tcb_chkpwd setuid helper lives in /usr/lib/chkpwd/ which is **Permission-denied to non-root**. - ⇒ pam_tcb running as a **non-root** process (our systemd --user service) CANNOT read the hash nor invoke the helper, so pam_authenticate returns PAM_AUTH_ERR for EVERY password (verified: wrong AND correct both → rc=7). Non-root PAM verify is impossible here. - Password login would therefore need a **root/system** auth service. Operator directive: “sudo can’t be used” + emphatic “**All access MUST BE configured to use ssh key(s)!!!**”.

Positive facts enabling the pivot: - ~/.ssh/authorized_keys readable (6 keys); ssh-keygen -Y sign/verify supported ⇒ ssh-key challenge-response works fully **as non-root milosvasic**, no password, nothing stored.

New login mechanism: SSH-key challenge-response

Service (rename services/pam_auth/ → services/auth_gate/, module digital.vasic.helixcode/auth_gate; keep ALL the cookie/session/rate-limit/HTTP machinery — only the Verifier + the two login endpoints change):

- **GET /login** → issue a fresh **challenge**: a random 32-byte nonce, HMAC-signed by the server (bind to a short TTL + client) so it can't be forged/replayed; render a login page showing the challenge + the EXACT command the user runs locally: `printf %s '<challenge>' | ssh-keygen -Y sign -n helixcode-login -f ~/.ssh/id_ed25519` → paste the signature.
- **POST /login** → form { principal, signature }. Verify:
 1. the challenge HMAC is valid + unexpired (server-issued, not replayed);
 2. `ssh-keygen -Y verify -n helixcode-login -f <allowed_signers> -I <principal> -s <sigfile>` succeeds over the challenge bytes, where <allowed_signers> is derived from milosvasic's `~/.ssh/authorized_keys` (each key → an allowed-signers line for the configured principal). On success → issue the existing signed session cookie; on failure → 401, generic error, rate-limited.
- **Verifier interface stays**; concrete becomes `sshSigVerifier` (uses `ssh-keygen -Y verify` against the allowed-signers file). The `pamVerifier` stub is removed.
- **GET /auth, POST /logout, cookie, rate-limit, fail-closed** — UNCHANGED from the built service.

Params: `HELIX_AUTH_MODE=sshkey`, `HELIX_AUTH_ACCOUNT=milosvasic`, `HELIX_AUTH_AUTHORIZED_KEYS` (default `~/.ssh/authorized_keys`), `HELIX_AUTH_PRINCIPAL` (default the account name). No password param anywhere.

Autonomous test path (real evidence, §11.4.98 — no operator, no password)

Tests generate a challenge, sign it with milosvasic's real key via `ssh-keygen -Y sign`, POST the signature → expect 302 + cookie; tamper the signature / use a non-authorized key / replay an old challenge → expect 401. All with captured evidence. This is MORE testable than PAM (no secret needed).

Reuse / churn

- Reused as-is: cookie HMAC + secret, session TTL, rate-limiter, /auth, /logout, fail-closed, Caddy forward_auth render, systemd --user units, install flow, TLS machinery.
- Changed: Verifier (PAM→ssh-key), /login GET+POST, service name (pam_auth→auth_gate), env params, docs wording (PAM→ssh-key), .env.example.

Credentials (§11.4.10)

The login stores NO password. The operator-provided milosvasic/su passwords are NOT used and NOT stored — the test secret is scrubbed to the account name only. ~/.ssh private keys are never read by the service (only authorized_keys/public material for verification).